

FDE — Candidate Project Challenge

Applied AI | phData

Welcome

You are going to play the role of a phData engineer building an agentic proof of concept for a fictional organization: Solstice Hotel Group. You will build an agentic system that will solve challenges for the hotel staff by building a front-end that guests can interface with and automate common tasks (details listed below).

During the project challenge interview, you will roleplay as a phData FDE and demo the solution to the interviewers (roleplaying as Solstice Hotel stakeholders). You will be expected to show the interviewers how the solution works at both a functional level (how does it solve their day-to-day problems) and technical level (how does it work under the hood). Interviewers will roleplay both a technical persona and a non-technical product owner from the business. Be prepared to answer questions from both personas.

Interviewers will want to see the following: how you creatively solve ambiguous business problems, how you build with agentic AI tools, how you architect for production, and how clearly you can explain what you did. There is no single "right" platform or solution.

The Scenario

Solstice Hotel Group runs 140 upper-midscale properties and has built its brand on treating guests like people, not folio numbers.

Two things are true right now:

- The front desk and central reservations line are buried in repetitive guest requests
- The group sales team is still building multi-room block quotes by hand in spreadsheets, so a request that should take twenty minutes takes two days.

Leadership wants an AI agent to help with both, and they want something that could plausibly ship next quarter. Interviewers will act as the director of engineering and a non-technical product owner. They will be vetting your proof-of-concept solution and determining if they will hire phData to implement and deploy their desired solution.

What You'll Build

- **Guest & Staff Concierge Agent (conversational).** Grounded in the provided guest, reservation, and property policy data, this side of the agent answers guest questions, handles routine requests (late checkout, amenity booking, loyalty status), and knows what it can't do — refunds past a threshold, disputes, anything medical or legal — and escalates with full context instead of a dead-end "please call us."
- **Group Booking Workflow (agentic).** When a group inquiry comes in, the agent should gather requirements, check availability and rate feasibility across the relevant properties, apply Solstice's group pricing rules, and generate a structured proposal — or flag it for a human when it falls outside those rules.
- **User Interface:** Create an interface that showcases how the end-users interact with the agents. The user interface should be usable by guests and allow for questions in natural language. **The interface cannot just be Claude or ChatGPT. Interviewers are looking for a custom fit-for-purpose interface.**

You'll also design at least one net-new tool or skill for the agent that isn't handed to you in the sample data — something you decide the agent needs based on the scenario. Same muscle as building an agent skill from scratch, just folded into one project rather than a separate one.

- **Data provided:**
 - The following CSV outputs represent sample exports from the Solstice database. The .txt file contains instructions to the front desk on hotel policies.
 - **solstice-guest-profiles.csv** — Guest and reservation records for the concierge team to track guest info, including loyalty status and front-desk notes.
 - **solstice-group-inquiries.csv** — Sample inbound group booking requests that come from the company website reservation portal.
 - **solstice-properties.csv** — A directory of Solstice properties with room inventory, rates, and group-booking thresholds to check requests against.
 - **solstice-property-policies.txt** — The front desk's plain-English policy reference for cancellations, escalations, and guest requests.

Requirements & Guardrails

- The concierge should resolve most routine guest questions without a human in the loop. Anything it hands off should hand off with full context, not a restart.

- The agent should never invent a policy, rate, or availability. If it isn't grounded in the provided data, it says so or escalates.
- Any group proposal outside standard rules (discount past a threshold, block size above a limit, blackout dates) gets flagged for a human, not auto-approved.
- Guest PII, especially payment info, should never sit unmasked in anything the agent can see or expose.
- Set your own target for response latency and be ready to justify it. We want to see you reason about a real production constraint, not guess a number.

What Else We Want From You

- **An architecture diagram (e.g., [Draw.IO](#), [Lucid Chart](#)).** Future state of the production-hardened solution running in production. Showcase the system diagram of how the solution could be deployed into production, note that your current solution can be assumed to be containerized/modified to fit an ideal state future architecture. Include where guest and reservation data comes from, where the agent runs, how it's monitored, how it degrades gracefully when something upstream fails.
 - **A short integration recommendation.** Spend a little time understanding how a hotel chain like this actually runs its technology today (property management system, central reservations, loyalty/CRM), and write a few paragraphs on how you'd realistically plug this in, and what you'd tell an IT team that's nervous about it. Bonus points if you address the very real fear at the front desk that "the AI is coming for our jobs" with something more useful than a platitude.
- **Be ready to explain and modify it.** Use AI tools to build fast, we expect that. In the follow-up conversation we'll ask you to open the hood and change something live, so make sure the understanding is yours, not just the typing.

Deliverables

- Source code, Agent configuration (.md): system prompt(s), tool/skill definitions, guardrails, and a few sample transcripts
- Architecture diagram (image or diagram file) showing the future-state production system
- Native export files from your platform of choice, if applicable (.zip, .json, etc.)

Rules of Engagement

Use any AI tool you want (Claude, ChatGPT, Copilot, Gemini, whatever). Use any agent platform you want (Copilot Studio, Snowflake Cortex, Bedrock, Vertex AI, n8n, Claude Projects, or something less common — just tell us why). If something in the brief is ambiguous, make an assumption and state it. That's a positive signal, not a gap.

Time & Submission

Estimated effort is 4 to 5 hours. You'll have 5 business days from receipt to submit. We'll send you the submission address with your challenge package.

How This Gets Evaluated

The file is half the picture. There are two follow-up conversations built around it:

- **A Business Conversation**, where you'll walk a non-technical stakeholder through your solution, take some pressure-testing questions, and show us you can build trust in a solution the way you would in front of a client.
- **A Technical Conversation**, where we dig into your architecture and your code, and ask you to change something on the spot.